Quel est l'impact de l'IA générative sur les méthodes formelles de vérification du logiciel

Mohamed Ali, Mosaab Seif El Islme Dehane, Khouloud Saadia Kaid, Thinhinane Ziane, Younes Benhamroura
UMLP, Besançon, France

16 avril 2025

Plan

1. Introduction
2. Contexte
3. État de l'art
4. Limites
5. Travaux à réaliser
6. Conclusion

1. Introduction

Méthodes formelles et IA générative

- Les méthodes formelles permettent de vérifier, avec des outils mathématiques, que les logiciels respectent bien ce qu'on attend d'eux.
- Elles sont très utilisées dans les domaines sensibles (avions, trains, hôpitaux), où les erreurs ne sont pas permises.
- Mais elles sont souvent *longues, difficiles à mettre en place*, et demandent beaucoup de connaissances techniques.
- Les modèles d'IA générative (comme ChatGPT, Claude, LLaMA) sont capables aujourd'hui de *générer du code* et *aider à vérifier des logiciels*.
- Ce projet de Master 2 propose d'étudier comment l'IA peut aider les humains à faire des preuves formelles, ou même les automatiser.
- **Objectif :** Mettre en commun les atouts de l'IA et des méthodes formelles pour rendre la vérification logicielle plus simple, rapide et accessible.

Problématique

Peut-on utiliser l'IA générative pour rendre ces méthodes plus accessibles ?

2. Contexte

Méthodes Formelles Classiques : Définition

Définition [CT21, JS24]

- Techniques mathématiques pour raisonner rigoureusement sur les systèmes
- Sémantique formelle + Logique mathématique
- Ex : Preuves Coq, Modèles SPIN, Contrats Why3

Motivations Clés [Ariane 501, A380]

- **Risques critiques** : Échec Ariane 5 (370M$)
- **Précision** : Éviter les overflows ($x = -2^{31}$)
- **Certification** : Logiciel A380 vérifié par Astrée

Défi Commun

$n = 100 \Rightarrow$ Plus d'états que d'atomes dans l'univers observable

Trois Paradigmes Fondamentaux

Vérification Déductive

- Coq/Isabelle
- Logique de Hoare
- seL4 kernel

Model Checking

- SPIN/NuSMV
- Protocoles NASA
- LTL/CTL

Test par Propriétés

- QuickCheck
- Génération aléatoire
- Invariants formels

Limitations Majeures

- Complexité exponentielle (2^n états)
- Expertise manuelle intensive

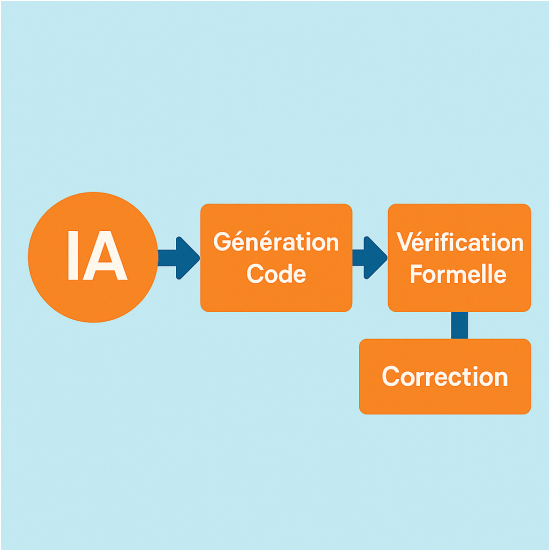
IA Générative : Nouveaux Paradigmes

4 Apports Clés [PS20, Team23]

1. Génération de code annoté (Codex)
2. Assistance aux preuves (GPT-f)
3. Optimisation SMT (SolSearch)
4. Extraction d'invariants (DeepSpec)

Défis Emergents

- Fiabilité (des erreurs 25% des cas)
- Coût énergétique (500 foyers/an)



Pipeline

IA-Formel
Generate $\longrightarrow$ Verify $\longrightarrow$ Repair

Cas Réel [DeepSeek-R1]

40% réduction d'étapes de preuve pour CompCert

3. État de l'art

Représentation des formules logiques

Objectif :

Permettre à l'IA de comprendre la logique mathématique

1. Approche syntaxique
L'IA lit la structure de la formule (arbre, symboles, profondeur, etc.)
Exemple : Représenter une formule comme un arbre syntaxique

2. Approche sémantique
Elle encode le sens implicite via des vecteurs (Word2Vec, LSA)
Exemple : Comprendre que $P(x) \to Q(x)$ est proche de $P(x) \vee Q(x)$

3. Approche neurale
Utilisation de réseaux neuronaux pour capturer des schémas logiques complexes
Exemple : GNN, Transformers, TreeNN

Résultat : Jusqu'à +60% de performance dans la résolution automatique

Percée dans la sélection des prémisses

Problème

Comment sélectionner 100 axiomes utiles parmi 100,000 disponibles ?

Évolution des méthodes :

- **k-NN** : Méthode simple (40–50% de précision)
- **Naïve Bayes** : Approche probabiliste
- **GNNs** : Jusqu'à 92% de précision

Impact :

- Réduction massive de l'espace de recherche ($\times$100–1000)
- Résolution de théorèmes auparavant insolubles

Conclusion : L'IA filtre efficacement l'information pour éviter l'explosion combinatoire

Aide à la preuve dans les assistants interactifs (ITP)

Contexte :
Dans des systèmes comme **Coq**, **Lean** ou **HOL4**, les utilisateurs construisent des preuves manuellement avec des tactiques.

Problème :
Long, complexe, demande une expertise importante

Apport de l'IA :

- Suggestion automatique de tactiques
- Génération de séquences de preuves

Exemples d'outils :

- **TacticToe** : dans HOL4
- **Tactician** : dans Coq

Bénéfice : Moins d'effort humain, preuves plus rapides

Approches hybrides : logique + neurones

Objectif

Combiner les forces du raisonnement logique symbolique et de l'intelligence neuronale.

Exemples de projets : :

- **NeuroSAT** : prédit la satisfiabilité de formules SAT
- **DeepProbLog** : fusion de Prolog et de réseaux neuronaux

Pourquoi c'est prometteur ?

- Ça permet de raisonner dans l'incertain
- L'IA peut apprendre à guider des preuves, ou compléter automatiquement des règles logiques

SolSearch : Une révolution IA dans la conception de solveurs SAT

Une approche récente et prometteuse

SolSearch [JS24] est le premier cadre à exploiter les LLMs pour générer, ajuster et optimiser automatiquement du code de solveur SAT.

Principe clé

- L'IA suit un plan d'apprentissage progressif (du plus simple au plus complexe)
- À chaque étape : elle choisit une partie du code, propose une version améliorée, puis teste le résultat
- Elle peut découvrir des techniques efficaces que les humains n'auraient pas imaginées

SolSearch : Gains mesurables et impact

Gains mesurables

- Des solveurs plus rapides et plus efficaces, comme CaDiCaL, Kissat et Z3
- Parfois jusqu'à 11% de gain de performance, et plus de problèmes résolus

Impact

- L'IA ne fait pas que générer du texte ou du code, elle peut aussi aider à concevoir des outils logiques
- Cette méthode peut aussi s'adapter à d'autres types de problèmes (comme MaxSAT ou QBF)

3. Limites actuelles

Limites 1 et 2 : Généralisation & Fiabilité

Généralisation

Les modèles sont souvent entraînés sur des bases spécifiques, comme Mizar ou HOL4. Donc, dès qu'on change de contexte logique ou de langage formel, ils perdent en efficacité.

- **Exemple** : Une preuve correcte en Coq peut échouer en Why3.
- **Conséquence** : Pas de transfert universel des capacités.

Fiabilité et "hallucinations logiques"

Les IA génératives, comme GPT-4, peuvent produire des preuves qui ont l'air crédible, mais qui sont totalement fausses.

- **Risque** : Ces erreurs peuvent être difficilement détectables.
- **Impact** : Vérification manuelle toujours nécessaire.

Limites 3 et 4 : Explicabilité & Intégration

Manque de traçabilité

Les modèles sont souvent des “boîtes noires” : on ne sait pas pourquoi ils sélectionnent telle clause ou telle tactique.

.**Difficulté** : Difficile de corriger ou comprendre les erreurs.

Intégration aux outils formels

Les IA ne sont pas encore intégrées nativement dans les environnements comme Why3, Coq ou Isabelle.

- **Conséquence** : L’utilisation reste expérimentale ou externe(La plupart du temps, les tests se font à l’extérieur. Il n’existe pas encore de vraie interface pratique et stable.)

Limites 5 et 6 : Preuves longues & Coût élevé

Complexité des preuves longues

Les modèles actuels échouent fréquemment sur des raisonnements complexes étalés dans le temps.

- **Observation** : Ces approches fonctionnent surtout sur des preuves courtes. Dès qu'une preuve devient plus longue ou plus complexe, les modèles échouent beaucoup plus souvent.

Coût de calcul

Les modèles (Transformers, GNNs) nécessitent beaucoup de puissance pour être entraînés et utilisés.

- **Conséquence** : Difficulté d'accès pour une utilisation quotidienne ou pédagogique.

4. Travaux à réaliser

Objectif du projet

But global

utiliser soit un modèle génératif, soit une base logique comme Prolog, pour aider à créer ou compléter des preuves.

- Deux axes d'expérimentation sont proposés.
- Ils s'appuient sur deux environnements bien connus : **Why3** et **Prolog**.

Option 1 : IA générative dans Why3

Objectif

Étudier comment un modèle génératif (comme GPT-4 ou Claude) peut suggérer ou générer une preuve complète ou partielle en WhyML.

Étapes :
On utilise un modèle comme GPT-4 dans Why3. On lui fournit des prompts pour qu'il propose une preuve ou un fragment de preuve.
Ensuite, on compare le résultat avec une preuve humaine. L'objectif est d'en faire un assistant dans Why3IDE, et voir s'il peut vraiment aider.
Résultat attendu :

- Évaluer l'utilité, la fiabilité et les limites de l'IA générative dans un environnement formel.

Option 2 : Raisonnement explicable avec Prolog

Objectif

Développer un moteur d'aide à la preuve en Prolog, basé sur des règles logiques simples et éventuellement enrichi d'une couche d'IA.

Étapes :
On construit un système logique simple en Prolog. On y ajoute une base de faits et des règles.
Puis, on crée un assistant de raisonnement qui peut proposer une chaîne logique. Et en bonus, on peut lui ajouter un petit modèle d'IA, comme GPT ou k-NN, pour guider la preuve.
Résultat attendu :

- Allier la rigueur logique de Prolog avec l'intuition statistique de l'IA.

5. Conclusion

Conclusion — Quel impact pour les méthodes formelles ?

Méthodes formelles classiques :
Puissantes et rigoureuses
Mais complexes, longues, réservées aux experts

Apport de l'IA générative :
Compréhension automatique, sélection intelligente, génération de preuves
Améliorations mesurées (ex : SolSearch, ENIGMA)
Risques de fiabilité, intégration encore limitée

Ce que nous avons appris :

- L'IA peut accompagner l'humain dans les preuves formelles
- Elle accélère, automatise partiellement, et rend certaines tâches plus accessibles
- Mais elle ne remplace pas la rigueur des démonstrations formelles

Perspectives :

- Intégrer l'IA dans Why3 ou Prolog pour un raisonnement hybride
- Développer des outils sûrs, explicables et intégrés

Conclusion finale

L'IA générative ne remplace pas les méthodes formelles, mais les complète.
Elle permet de construire des outils de vérification plus rapides, plus fiables et plus accessibles.